

Prowadzący przedmiot: Dr. Inż Sławomir Bujnowski

Student: Jakub Ćwiora

Specyfikacja

1. Oprogramować komunikację mikroprocesora STM32F103RB z PC poprzez interfejs asynchroniczny z wykorzystaniem przerwań i buforów kołowych.
2. Zaprojektować i zaimplementować protokół komunikacyjny pozwalający na:
 - Adresowanie ramek
 - Przekazywanie dowolnych danych
 - Weryfikację poprawności przesłanych danych z uwzględnieniem ich kolejności
3. Obsługa przetwornika cyfrowo analogowego DMA z buforem kołowym o długości 512 słów
 - Obsługa przerwań w połowie i na końcu bufora
 - Częstotliwość próbkowania 8kHz
 - Dla każdych 256 próbek liczyć Dyskretną Transformatę Fouriera
 - Umożliwić odczyt amplitud dla poszczególnych prążków

Protokół Komunikacyjny

Początek ramki	Nadawca	Odbiorca	Identyfikator danych	Dane	Suma kontrolna	Koniec ramki
(	XX	XX	Dowolne zakodowane wartości			)
0x28	Dowolne nie zawierające 0x28 i 0x29**	Dowolne nie zawierające 0x28 i 0x29**				0x29
1 bajt	2 bajty	2 bajty	1 (2)* bajt	1-120 (2-240)* bajtów	2 (4)* bajty	1 bajt

* Wartości w nawiasach to ilość bajtów po zakodowaniu

** W przypadku wystąpienia takiego znaku ramka nie zostanie poprawnie rozpoznana

Minimalna długość = 14

Maksymalna długość = 254

Nadawca / Odbiorca

PC (0x50 0x43) – komputer

ST (0x53 0x54) – płytką STM32F103RB

Kodowanie pól

Pola identyfikatora danych, danych i sumy kontrolnej są kodowane w systemie szesnastkowym.

Surowe bajty są przekształcane na reprezentacje w postaci dwóch bajtów z zakresu [0x30, 0x39] ∪ [0x41, 0x46], występują zera wiodące.

Dane

Muszą być parzyste z racji na kodowanie szesnastkowe, gdy tak nie jest zwracany jest błąd ERR_PROTO – bo prawdopodobnie coś poszło nie tak w kodowaniu do wysłania. Następnie są rozkodowywane, jeśli wystąpi tu niepożądany znak (z poza zakresu [0x30, 0x39] U [0x41, 0x46]) to zwracany jest błąd ERR_PROTO. Gdy dane przejdą te procesy są przetwarzane przez komendy które interpretują podane liczby jako zapisane w Big Endian i dokonują konwersji na Little Endian by wykonywać działania arytmetyczne.

Identyfikator danych

To pole pozwala na rozróżnianie komunikatów używając do tego 2 bajtów (po zakodowaniu). Założyłem że wartości od 0x00 do 0x63 będą możliwymi poleceniami, 0x64 do 0xC7 odpowiedziami na nie i 0xC8 do 0xFF zarezerwowałem dla błędów.

Wartość	Nazwa	Długość pola Dane	Opis
0x00	GET_DFT_ON_INDEX	1	Parametrem jest indeks którego wartość chcemy odczytać podany jako jeden bajt
0x01	GET_DFT_IN_RANGE	2	Parametrem są indeksy pomiędzy których chcemy odczytać wartości. Podajemy je jako 2 bajty, gdzie pierwszy jest początkiem a drugi końcem zakresu
0x64	RSP_DFT_ON_INDEX	3	Pierwszym bajtem odpowiedzi jest indeks a drugi i trzeci bajt odpowiedzi to wartość //nie ma napisane czy to liczba 16-bitowa
0x65	RSP_DFT_IN_RANGE	Max 20	Wysyłane jest kilka ramek w sekwencji zawierających maksymalnie 8 kolejnych wartości z zadanego zakresu (w przypadku wysyłania niż 8 wartości, puste miejsca są wypełnione zerami). Bajty mają odpowiednie znaczenie: 1. Ilość ramek w sekwencji 2. Numer ramki w sekwencji 3. Ilość wartości w tej ramce (max. 8) 4. Pierwszy indeks w tej ramce 5-20. Wartości amplitud
0xC8	ERR_PARAM	1	Błąd Wartości Parametru
0xC9	ERR_CRC	1	Błąd Sumy Kontrolnej
0xCA	ERR_UNKNOWN	1	Nieznany identyfikator
0xCB	ERR_PROTO	1	Błąd przy przetwarzaniu ramki

Przykładowe odpowiedzi

Wartość	Ramka	Odpowiedź
0x00	(PCST000ABC45) Zapytanie o wartość na 10. prążku	(STPC640A00058087) - Jest komunikatem 0x64 - Pytanie jest o prążek numer 0x0A - Wartość na tym prążku to 0x0005 (Big Endian)
0x01	(PCST01040FC687) Zapytanie o wartości na prążkach [4,15]	(STPC6502010804000F0000000060005000400030000000712DE)(STPC650202040C0000000200000000E585) Pierwsza ramka: - Jest komunikatem 0x65 - W sekwencji są 0x02 ramki - Ta ramka jest 0x01 elementem sekwencji - W tej ramce są 0x08 amplitudy - Pierwszy prążek w tej ramce to 0x04 - Wartości na prążkach... Druga ramka: - Jest komunikatem 0x65 - W sekwencji są 0x02 ramki - Ta ramka jest 0x02 elementem sekwencji - W tej ramce są 0x04 amplitudy - Pierwszy prążek w tej ramce to 0x0C - Wartości na prążkach...

Suma kontrolna – CRC16/CCITT-FALSE

Jest obliczana lokalnie dla komendy z pola dane o podanej długości, a następnie porównywana z przysланą sumą kontrolną by sprawdzić poprawność.

Wybrany wielomian: 0x1021

Wartość początkowa: 0xFFFF

Każdy nowy przetwarzany bajt jest przesuwany w lewo o 8 bitów i poddawany operacji XOR z dotychczasowo obliczoną sumą kontrolną. Następnie osiem razy następuje operacja przesuwania o jeden bit w lewo, w przypadku gdy najbardziej znaczący bit jest równy 1 – po przesunięciu wartość sumy jest poddawana operacji XOR z wielomianem 0x1021. Po tych działaniach pobierany jest kolejny bajt.

Przesyłana jest jako 4 bajty które poddane zamieniane zostają na jedną liczbę 16 bitową:

28 29 staje się 32 38 32 39

Uniemożliwia to wystąpienie zakazanego znaku w tym polu (np. dla sumy równej 0x2829 występują znaki początku i końca ramki)

Obsługa błędów

- **Wiele znaków początku ramki.** Z każdym znakiem początku ramki rozpoczyna się czytanie ramki na nowo – poprzednie wartości są zapominane.
- **Wiele znaków końca ramki.** Wszystkie znaki końca ramki są ignorowane, jeżeli wcześniej nie wystąpił znak początku ramki.
- **Znak początku ramki po wykryciu początku ramki.** Rozpoczęcie procesu czytania ramki od nowa.
- **Nie wykrycie znaku końca ramki, po znaku początku ramki.** Nie wykryto żadnej ramki, więc nic się nie dzieje. Jeśli wystąpił znak początku ramki i jej długość przekroczyła maksymalną – wysyłany jest komunikat ERR_PROTO.
- **Nieprawidłowy odbiorca.** Ramka po odbiorze zostaje zignorowana w całości.
- **Nieprawidłowa suma kontrolna.** Wysyłany jest komunikat, że podana i wyliczona suma kontrolna są różne – ERR_CRC. Jest to komunikat z priorytetem wyższym od komunikatu o *nieznanym identyfikatorze danych*.
- **Nieznany identyfikator danych.** Wysyłany jest komunikat ERR_UNKNOWN.

Obliczanie DFT (FFT)

Szerokość jednego prążka (czyli rozdzielczość częstotliwościową) liczymy za pomocą wzoru:

$$\Delta f = \frac{f_s}{N}$$

Gdzie:

f_s - Częstotliwość próbkowania

N – Ilość próbek

Dla naszego przypadku:

$$\Delta f = \frac{8000Hz}{256} = 31,25Hz$$

Liczymy z 256 próbek, gdyż bufor ma 512 miejsc, a czytamy z niego w połowie i na końcu.

Do liczenia transformaty używamy funkcji z biblioteki CMSIS-DSP o nazwie `arm_cfft_q31` która wykorzystuje liczby stałoprzecinkowe ze względu na brak FPU w mojej płytce. Używamy formatu 32 bitowego, by mieć większą czułość na zmiany.

Dane z czujnika są na początku zmniejszane o średnią z otrzymanego sygnału, gdyż chce żeby wartości oscylowały wokół zera, następnie przesuwane o 14 bitów w lewo dla większej czułości i zapisywane w formacie stałoprzecinkowym *q31_t*. Takie wartości przepisywane są do tablicy o rozmiarze 512 słów, ponieważ co każdą rzeczywistą wartość z czujnika musimy podać część urojoną aby funkcja *arm_cfft_q31* mogła poprawnie zadziałać.

Wynikową tablicę o długości 512 słów musimy przetworzyć na amplitudy z użyciem wbudowanej funkcji *arm_cmplx_mag_q31* by otrzymać 256 amplitud wynikowych (warto pamiętać że od 128 indeksu, prążki będą powielone lustrzanie) gdzie każda odpowiada prążkom których środki występują co 31.25 Hz. Wartości tych amplitud są poddawane normalizacji, tj. przesunięciu o 12 bitów w prawo by mogły być zapisane jako liczba całkowita 16-bitowa bezznakowa.

Opis obsługi czujnika Grove – Light Sensor 1.2 z chipem LM358

W projekcie wykorzystałem moduł czujnika natężenia światła, opartego na fotorezystorze LS06-S oraz wzmacniaczu operacyjnym LM358. Układ ten pełni rolę przetwornika rezystancji na napięcie. Zgodnie ze specyfikacją, rezystancja fotorezystora maleje wraz ze wzrostem natężenia światła, co przekłada się na wzrost napięcia wyjściowego sygnału analogowego.

Czujnik wymaga podpięcia do 5V, co oznacza że sygnał wyjściowy może osiągnąć zbliżoną do niego wartość – a pin obsługujący przetwornik ADC na płytce STM32 NUCLEO F103RB nie toleruje napięć wyższych niż 3.3V. By obejść ten problem należy zastosować dzielnik napięcia, lub podpiąć czujnik do 3.3V i stracić ~35% wykrywanego zakresu.

Przetwornik ADC jest 12-bitowy, więc wynik to liczba z zakresu od 0 do 4095 (2 do potęgi 12 = 4096 wartości). Jeśli na wejściu przetwornika pojawi się napięcie V_{ADC} , to powinniśmy odczytać wartość:

$$ADC = V_{ADC} / 3,3 \text{ V} * 4096$$

Stąd wynika też:

$$V_{ADC} = ADC * V_{czujnika} / 4096$$

Proponycja rezystancyjnego dzielnika napięcia składającego się z rezystorów:

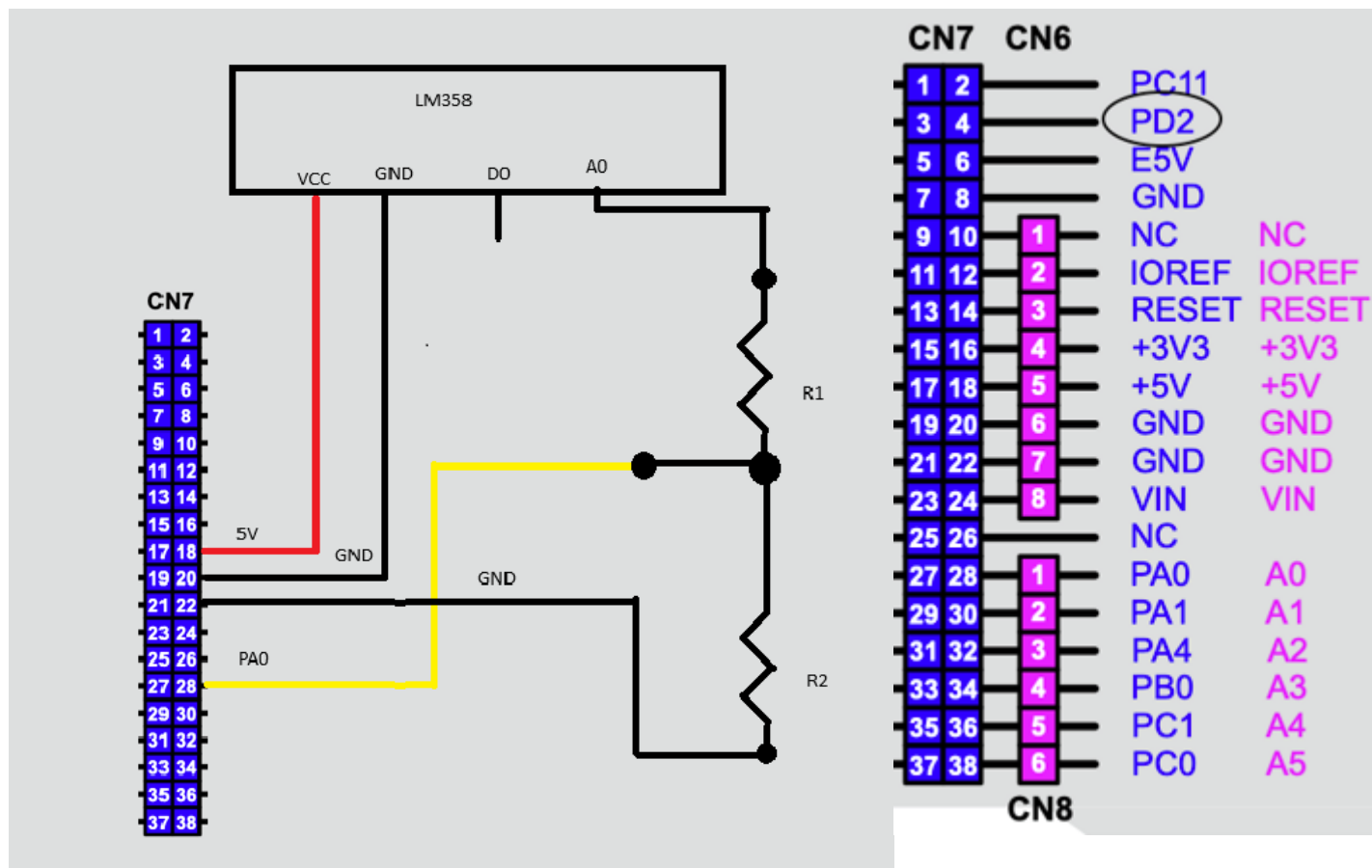
$$R_1 = 2.2 \text{ k}\Omega, R_2 = 3.3 \text{ k}\Omega$$

Wtedy maksymalne napięcie wejściowe wynosić będzie:

$$V_{in} = \frac{V_{sensor} * R_2}{R_1 + R_2}$$
$$V_{in} = \frac{5.0V * 3.3k\Omega}{2.2k\Omega + 3.3k\Omega} \approx 3.0V$$

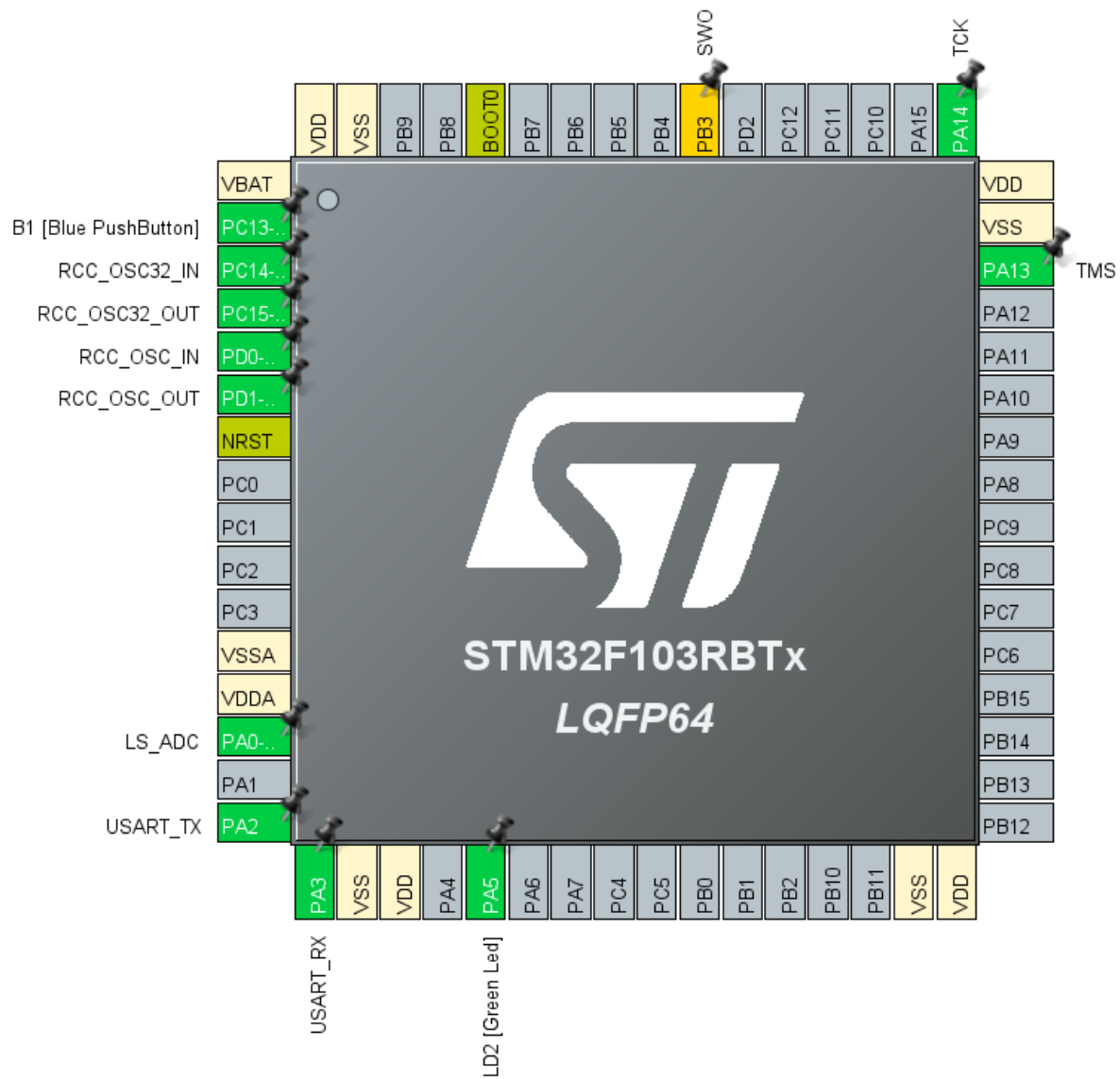
Co zapewni bezpieczne napięcie z marginesem błędu 0.3V dla ADC.

Schemat podłączenia



Sygnal analogowy pobierany jest przez przetwornik ADC1 na kanale zerowym który znajduje się na pinie PA0.

Konfiguracja w CubeMX



ADC1

Parameter Settings

User Constants

NVIC Settings

Configure the below parameters :

Search (Ctrl+F)

ADCs_Common_Settings

ModeIndependent mode

ADC_Settings

Data AlignmentRight alignment

Scan Conversion ModeDisabled

Continuous Conversio...Enabled

Discontinuous Conver...Disabled

ADC_Regular_ConversionMode

Enable Regular Conve...Enable

Number Of Conversion1

External Trigger Conv...Timer 3 Trigger Out event

Rank1

ADC_Injected_ConversionMode

Enable Injected Conve...Disable

WatchDog

Enable Analog Watch...☐

User Constants

NVIC Settings

DMA Settings

GPIO Settings

Parameter Settings

DMA Request	Channel	Direction	Priority
ADC1	DMA1 Channel 1	Peripheral To Memory	Low

Add

Delete

DMA Request Settings

ModeCircular

Increment Address☐

Data WidthHalf Word

Peripheral

Memory☒

Half Word

ADC1 Mode and Configuration	
Mode	
☒	INO

TIM3

TIM3 Mode and Configuration

Mode

Slave Mode Disable ▼

Trigger Source Disable ▼

Clock Source Internal Clock ▼

Channel1 Disable ▼

Channel2 Disable ▼

Channel3 Disable ▼

Channel4 Disable ▼

Combined Channels Disable ▼

☐ Use ETR as Clearing Source

☐ XOR activation

☐ One Pulse Mode

Configuration

Reset Configuration

✔ Parameter Settings

✔ User Constants

✔ NVIC Settings

✔ DMA Settings

Configure the below parameters :

⏪
⏩
i

▼ Counter Settings

Prescaler (PSC - 16 bits value) 7

Counter Mode Up

Counter Period (AutoReload Regist... 999

Internal Clock Division (CKD) No Division

auto-reload preload Enable

▼ Trigger Output (TRGO) Parameters

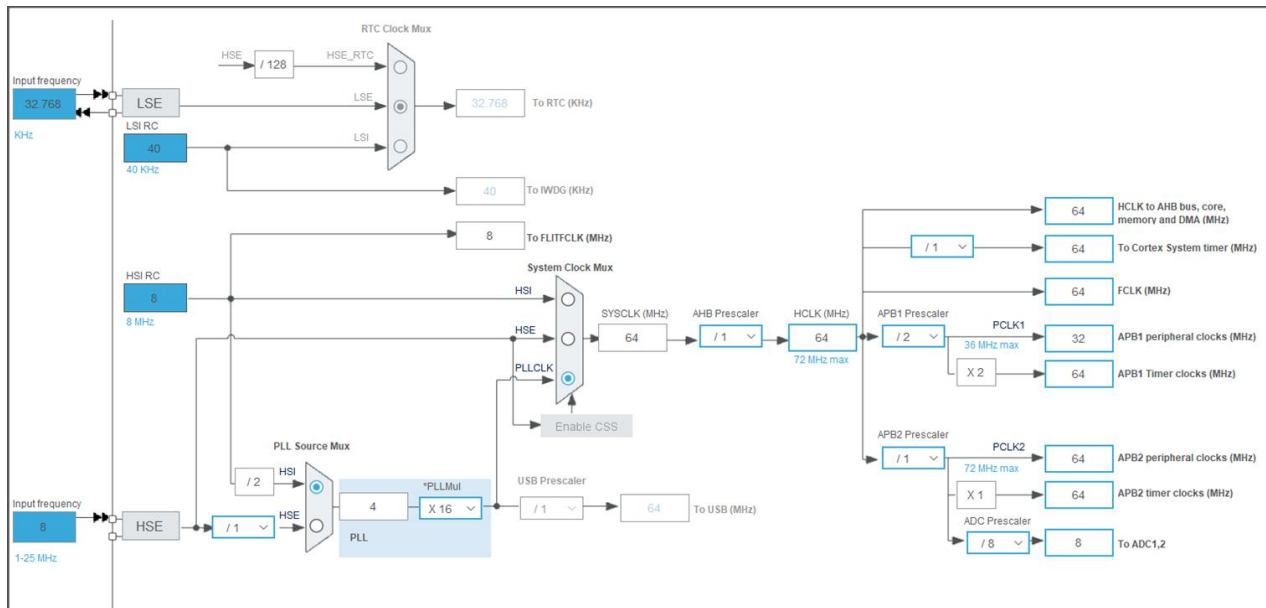
Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)

Trigger Event Selection Update Event

$$UpdateEvent = \frac{f_{clk}}{(PSC + 1) * (ARR + 1)}$$

$$UpdateEvent = \frac{64MHz}{(7 + 1) * (999 + 1)} = 8kHz$$

Clock Configuration



USART2

USART2 Mode and Configuration

Mode

Mode

Hardware Flow Control (RS232)

Configuration

Reset Configuration

☒ User Constants

☒ NVIC Settings

☒ DMA Settings

☒ GPIO Settings

☒ Parameter Settings

Configure the below parameters :

Basic Parameters

Baud Rate115200 Bits/s

Word Length8 Bits (including Parity)

ParityNone

Stop Bits1

Advanced Parameters

Data DirectionReceive and Transmit

Over Sampling16 Samples

Configuration

Reset Configuration

☒ Parameter Settings

☒ User Constants

☒ NVIC Settings

☒ DMA Settings

☒ GPIO Settings

DMA Request	Channel	Direction	Priority
USART2_RX	DMA1 Channel 6	Peripheral To Memory	Low

Add

Delete

DMA Request Settings

Mode

Increment Address☐

Peripheral☐

Memory☒

Data Width

NVIC

✓ NVIC
✓ Code generation

Priority Group .. ▾
☐ Sort by Preemption Priority and Sub Priority
☐ Sort by interrupts

Search ↺
Show available interrupts ▾
☒ Force DMA chan

NVIC Interrupt Table	Enabled	Preemption Priority
Non maskable interrupt	☒	0
Hard fault interrupt	☒	0
Memory management fault	☒	0
Prefetch fault, memory access fault	☒	0
Undefined instruction or illegal state	☒	0
System service call via SWI instruction	☒	0
Debug monitor	☒	0
Pendable request for system service	☒	0
Time base: System tick timer	☒	0
PVD interrupt through EXTI line 16	☐	0
Flash global interrupt	☐	0
RCC global interrupt	☐	0
DMA1 channel1 global interrupt	☒	0
DMA1 channel6 global interrupt	☒	0
ADC1 and ADC2 global interrupts	☒	0
TIM3 global interrupt	☐	0
USART2 global interrupt	☒	0
EXTI line[15:10] interrupts	☒	0